

DocMind to Production

The Container

System Design Notes — 2
Docker + Git MLOps

Friday, April 3

Docker is the single most important prerequisite for everything AWS-related. Your app runs inside a sealed, reproducible container that behaves identically on your laptop, your colleague's machine, and AWS.

PART 1

Git for MLOps

Branching · Secrets · Tags

PART 2

Docker Concepts

Images · Layers · Compose

PART 3

Implementation

Step-by-step build guide

Part 1: Git for MLOps

1.1 What is Git?

Git is a **version control system** that tracks every change you make to your code. Every meaningful change is saved as a **commit** — a named snapshot. Think of it as Google Docs' version history, but for your entire codebase.

Analogy — the Novel Writer's Filing Cabinet

You're writing a novel. After every chapter, you photocopy everything and drop it in a folder labeled with today's date. If you write a terrible chapter next week, you open the old folder and restore from there.

Git is that folder system — automated, instant, and for code. The folder labels are called commits.

1.2 Why MLOps Needs a Different Git Approach

A regular project has one artifact: code. An ML project has four, and only one of them belongs in Git:

Artifact	Goes in Git?	Reason
Python code (.py files)	YES ✓	Small, text-based, meaningful diffs
requirements.txt	YES ✓	Defines the reproducible environment
Dockerfile	YES ✓	Infrastructure as code — version it
.env (real API keys)	NEVER ✗	One mistake = public keys, hacked accounts, huge bills
PDFs / datasets	NO ✗	Too large; use S3 or DVC instead
Model weights (.bin, .pt)	NO ✗	Gigabytes in size; breaks GitHub repos
.env.example	YES ✓	Safe template with placeholders, no real secrets
docker-compose.yml	YES ✓	Local dev environment definition

1.3 Branching Strategy for ML Projects

A **branch** is an independent copy of your code where you can make changes without affecting the main working version.

Analogy — Recipe Testing

You have a working recipe (main branch). You want to try adding chilli. Don't ruin the original — write it on a separate card (new branch), test it, and if it's better, copy those changes back to the original.

If the chilli version is terrible, throw that card away. The original is untouched.

The three branches you need

Branch	Purpose	Rule
main	Production-ready code only. What gets deployed to AWS.	Only merge here when tested and working. GitHub Actions watches this branch.
dev	Active development. Day-to-day commits go here.	Your working branch. Commit here freely.
feature/xxx	One specific feature or experiment.	feature/async-endpoints, feature/ragas-eval, feature/dockerize

Daily workflow

```
# Start from dev
git checkout dev

# Create feature branch
git checkout -b feature/dockerize

# Work, then commit often
git add .
git commit -m 'feat: add Dockerfile with layer caching'

# When feature is done, merge back to dev
git checkout dev
git merge feature/dockerize

# When dev is stable and tested, merge to main
git checkout main
git merge dev
```

Why this matters for CI/CD (Day 4)

Your GitHub Actions pipeline will watch the main branch. When code lands on main, it automatically builds a Docker image and deploys to AWS ECS.

By keeping main clean and only merging tested code, you ensure only working code ever reaches production.

1.4 Tagging Releases

A **tag** is a named bookmark on a specific commit. Instead of saying “commit a3f9c2b was stable”, you say `v1.0`. Your Docker images will carry the same version tags, so you always know which code is running in which container.

```
# Tag your current commit
git tag -a v1.0 -m 'Dockerized. Health endpoint. Compose working.'

# Push the tag to GitHub
git push origin v1.0

# List all tags
git tag

# Restore a tagged version
git checkout v1.0
```

Tag after every major milestone: `v1.0` after today (dockerised), `v1.1` after Langfuse (Day 7), `v1.2` after AWS deployment (Day 9).

1.5 Secrets Management — The Most Important Rule

Secrets are API keys, passwords, database URLs — anything that grants access to a paid service. If a secret lands in a public GitHub repo, automated bots scan for them within minutes.

Real incident

A developer accidentally committed their AWS access key to GitHub. Within 11 minutes, bots found it and spun up 100 GPU instances to mine cryptocurrency. The AWS bill: \$50,000 in one weekend.

This is not hypothetical. It happens constantly. The prevention takes 5 minutes.

The three-file pattern you must follow

`.env` — real secrets, on your laptop only, NEVER committed:

```
OPENAI_API_KEY=sk-proj-abc123realkey...
PINECONE_API_KEY=pcsk-xyz789realkey...
LANGFUSE_SECRET_KEY=sk-lf-realkey...
```

.env.example — safe placeholder template, committed to the repo:

```
OPENAI_API_KEY=your_openai_api_key_here
PINECONE_API_KEY=your_pinecone_api_key_here
PINECONE_INDEX=your_index_name
LANGFUSE_PUBLIC_KEY=your_langfuse_public_key
LANGFUSE_SECRET_KEY=your_langfuse_secret_key
LANGFUSE_HOST=https://cloud.langfuse.com
APP_ENV=development
```

.gitignore — tells Git which files to completely ignore:

```
# Secrets - NEVER commit these
.env

# Python artifacts
__pycache__/
*.pyc
*.pyo
.pytest_cache/

# Virtual environments
venv/
.env/
env/

# ML artifacts - too large for Git
*.bin
*.pt
*.pkl
models/
data/

# OS files
.DS_Store
Thumbs.db
```

Part 2: Docker — Everything You Need to Know

2.1 The Problem Docker Solves

You've seen this before: code that works perfectly on your machine breaks on someone else's, or on a server. The reason is always the same: your code depends on an entire **environment** — Python version, library versions, OS settings — not just the code itself.

Analogy — The Vacuum-sealed Kitchen

A chef vacuum-seals their entire kitchen — every pan, spice, appliance — into a single box and ships it. Whoever opens it gets an identical kitchen. Every meal cooked is exactly the same.

Docker does this for software. It packages your code AND its entire environment into a sealed box called a container. Wherever that container runs — your laptop, AWS, your teammate's Mac — it behaves identically.

2.2 The Three Core Concepts

The Dockerfile — the recipe

A Dockerfile is a plain text file with step-by-step instructions for building your container. Docker reads the recipe and produces a result. If you give the same Dockerfile to anyone in the world, they get the identical result.

The Image — the finished product

When Docker follows the Dockerfile, it produces an **image**. The image is a frozen, immutable snapshot of your application and its complete environment. Images are stored in registries (DockerHub, AWS ECR) and can be downloaded and run anywhere.

The Container — the running instance

A **container** is a running instance of an image. You can run many containers from one image simultaneously, just as you can bake many cakes from one recipe. Containers are isolated from each other — this is why your 3 ECS instances don't interfere.

Term	Kitchen analogy	Technical meaning
Dockerfile	Recipe card	Text file with step-by-step build instructions
Image	Finished cake (not yet eaten)	Immutable, frozen snapshot of app + environment

Container	A slice of cake being eaten	Running instance of an image
Registry (ECR/DockerHub)	Bakery storage room	Remote storage where images are shared and pulled from
docker build	Following the recipe	Command to create an image from a Dockerfile
docker run	Serving a slice	Command to start a container from an image

2.3 Layer Caching — Why Docker Builds Are Fast (When Done Right)

A Docker image is not one monolithic blob. It's a stack of **layers**, each one representing one instruction in your Dockerfile. Docker caches these layers independently.

The tracing paper analogy

Imagine a stack of tracing paper. Bottom sheet: base Python. Next: system libraries. Next: your Python packages. Top: your app code.

If you change only the top sheet (your code), Docker doesn't redo the layers below. It reuses the cached layers and only rebuilds from the changed layer upward.

First build: 3 minutes. Every subsequent build where only your code changed: 10 seconds.

The golden rule: things that change RARELY go at the top; things that change OFTEN go at the bottom

```
FROM python:3.11-slim      # LAYER 1: base OS - changes almost never
WORKDIR /app              # LAYER 2: working directory - never changes
COPY requirements.txt .    # LAYER 3: dep list - changes rarely
RUN pip install -r req...  # LAYER 4: install deps - CACHED if req.txt
                           # unchanged
COPY . .                  # LAYER 5: your code - changes frequently
                           # (that's fine)
CMD ["uvicorn", "main:app"] # LAYER 6: start command
```

Why order matters: if you put `COPY . .` before `pip install`, every single code change invalidates the pip install layer and forces a full reinstall of all packages. That's 2–3 minutes per build instead of 10 seconds.

2.4 The Dockerfile for DocMind — Every Line Explained

```

# 1. Base image: Python 3.11 'slim' variant.
#   'slim' removes dev tools and docs, shrinking the image from ~900MB
#   to ~130MB.
#   Always use slim for production containers.
FROM python:3.11-slim

# 2. Set working directory inside the container.
#   All subsequent commands run from /app. Like doing 'cd /app'.
#   Best practice: always set WORKDIR explicitly.
WORKDIR /app

# 3. Copy ONLY requirements.txt first (not your code).
#   This is the layer caching trick. If requirements.txt hasn't changed,
#   Docker skips the pip install step entirely on the next build.
COPY requirements.txt .

# 4. Install Python dependencies.
#   --no-cache-dir: don't store pip's download cache inside the image.
#   This saves ~50MB of unnecessary image size.
RUN pip install --no-cache-dir -r requirements.txt

# 5. NOW copy your application code.
#   This layer changes frequently, but pip install above is already
#   cached.
#   The two-step COPY is what makes rebuilds fast.
COPY . .

# 6. Document that the container listens on port 8000.
#   This is metadata only - doesn't actually open the port.
#   You open the port with -p 8000:8000 when running the container.
EXPOSE 8000

# 7. The command to run when the container starts.
#   CRITICAL: --host 0.0.0.0 means 'accept connections from outside the
#   container'.
#   Without this, only code INSIDE the container can reach the API.
#   The load balancer and your browser are OUTSIDE the container.
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

2.5 The .dockerignore File

Just like `.gitignore` tells Git which files to ignore, `.dockerignore` tells Docker which files to exclude when copying your project into the image.

Security warning

Without `.dockerignore`, the `COPY . .` instruction copies your `.env` file (with real API keys) into the image. If you ever push that image to DockerHub (even accidentally), your keys are exposed to the world.

Always `.dockerignore` your `.env` file. This is non-negotiable.

```
# Secrets - never bake these into an image
.env

# Python bytecode - regenerated automatically
__pycache__
*.pyc
*.pyo
*.pyd

# Virtual environments - deps installed fresh via requirements.txt
venv/
.venv/

# Git history - not needed inside the container
.git
.gitignore

# Test and documentation files - keep the image lean
tests/
*.md
.pytest_cache/

# OS files
.DS_Store
Thumbs.db

# Docker files themselves don't need to be inside the image
docker-compose.yml
Dockerfile
```

2.6 Docker Compose — Running Multiple Services Together

DocMind doesn't run alone. In production it needs Redis. During development you might add a local Postgres. Starting and connecting these manually is tedious and error-prone.

docker-compose lets you define your entire multi-service application in one file. One command — **docker-compose up** — starts everything with the right configuration.

The Band Rider Analogy

A music band's rider says: "We need a stage, a sound system, 6 microphones, catering, and 4 hotel rooms." One document specifies everything needed.

docker-compose.yml is your application's rider. It says: "This app needs FastAPI on port 8000, Redis on port 6379, and they can talk to each other via the docmind-network."

```
version: '3.8'

services:

  # — Your FastAPI application —————
  app:
    build: .                                # Build image from Dockerfile in current
    dir                                         dir
    ports:
      - "8000:8000"                          # host_port:container_port
    env_file:
      - .env                                  # Load all secrets from .env
    automatically                             automatically
    volumes:
      - ./app                                # Mount code - file changes reflect
    instantly                                  instantly
                                              # (without rebuilding the image)
    depends_on:
      - redis                                # Wait for Redis to be ready before
    starting
    networks:
      - docmind-network

  # — Redis cache (shared across all app instances) —————
  redis:
    image: redis:7-alpine                    # Official Redis image - no Dockerfile
    needed
    ports:
      - "6379:6379"                          # Expose for local debugging tools
    networks:
      - docmind-network

# Internal Docker network so services discover each other by name.
# Your app code connects to Redis with: redis://redis:6379
# The hostname 'redis' is the service name defined above.
networks:
  docmind-network:
    driver: bridge
```

Command	What it does
docker-compose up	Start all services. Streams logs to terminal.
docker-compose up -d	Start all services in background (detached).
docker-compose down	Stop and remove all containers.
docker-compose logs app	See logs from the app service only.
docker-compose build	Rebuild images (run after changing Dockerfile or code).
docker-compose ps	List all running compose services and their status.

2.7 Essential Docker Commands Reference

Command	What it does
docker build -t docmind:v1 .	Build image from Dockerfile in current dir. Tag as docmind:v1
docker run -p 8000:8000 docmind:v1	Start container. Map laptop:8000 to container:8000
docker run -d --env-file .env -p 8000:8000 docmind:v1	Run in background with secrets from .env
docker ps	List all running containers
docker ps -a	List all containers including stopped ones
docker logs <container_id>	See stdout output from a container
docker exec -it <container_id> bash	Open a terminal INSIDE a running container
docker stop <container_id>	Stop a container gracefully
docker images	List all images on your machine
docker system prune	Delete stopped containers and unused images (free disk)

Port mapping explained

docker run -p 8000:8000 means: map port 8000 on my laptop to port 8000 inside the container.

The format is always host_port:container_port. The app inside always listens on 8000 (as per your CMD). You control what your laptop exposes.

If you used -p 9000:8000, you'd visit localhost:9000 in your browser and Docker forwards it to port 8000 inside the container.

Part 3: Implementation Guide

Before you start

Install Docker Desktop from docker.com/products/docker-desktop (free). The whale icon in your menu bar = Docker is running.

Verify: open terminal and run `docker --version`. You should see a version number like Docker version 25.x.x.

Navigate to your DocMind project folder in the terminal before starting.

Implementation 1 — Git Setup

Create and switch to a feature branch

1

```
# Make sure you're on dev first
git checkout dev

# Create today's feature branch
git checkout -b feature/dockerize

# Verify you're on the right branch
git branch    # the asterisk shows current branch
```

Create your .gitignore

Create a file named exactly `.gitignore` (with the dot prefix) in your project root. Add the content from Part 1, Section 1.5.

2

```
# Quick verify - .env should NOT appear here
touch .env    # create a test .env if you don't have one
git status
# If .env appears in git status, .gitignore isn't working
```

Create your .env.example

Create `.env.example` with all the variables your app needs but with placeholder values only. Use the template from Part 1, Section 1.5.

3

```
# Commit these safely
git add .gitignore .env.example
```

```
git commit -m 'chore: add gitignore and env template for MLOps'
```

Implementation 2 — The Dockerfile

Ensure you have a requirements.txt

1

```
# If you don't have one, generate it from your active environment
pip freeze > requirements.txt

# Verify it contains your key packages
cat requirements.txt | grep -E 'fastapi|langchain|pinecone|openai'
```

Create the Dockerfile

Create a file called exactly **Dockerfile** (capital D, no extension) in your project root. Paste this content:

2

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Check your file name

If your FastAPI app is in app.py and the instance is called api, change the last line to:

```
CMD ["uvicorn", "app:api", "--host", "0.0.0.0", "--port", "8000"]
```

Build the image

3

```
# Run from your project root (where Dockerfile lives)
```

```
docker build -t docmind:v1 .

# You'll see each layer being built:
# [1/6] FROM python:3.11-slim           (downloads base)
# [2/6] WORKDIR /app                     (instant)
# [3/6] COPY requirements.txt .           (instant)
# [4/6] RUN pip install --no-cache-dir ... (2-3 minutes)
# [5/6] COPY . .                         (instant)
# [6/6] CMD ...                          (instant)

# First build: 2-3 minutes. After that, ~10 seconds if only code changed.
# Should end with: Successfully built abc123def456
```

Implementation 3 — The .dockerignore

Create .dockerignore in the project root

Create the file and paste the content from Part 2, Section 2.5.

1

```
# Verify the .env file is excluded from the image
# Build the image and check what's inside
docker build -t docmind:v1 .
docker run --rm docmind:v1 ls /app

# You should NOT see .env in the output
# You SHOULD see main.py, requirements.txt, etc.
```

Implementation 4 — Run and Test

Add a /health endpoint to main.py (if you don't have one)

1

```
import time
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
async def health_check():
```

```
return {
  "status": "healthy",
  "service": "docmind",
  "version": "1.0",
  "timestamp": time.time()
}
```

Rebuild and run the container

2

```
# Rebuild with new health endpoint
docker build -t docmind:v1 .

# Run with your secrets
docker run -p 8000:8000 --env-file .env docmind:v1

# Expected output:
# INFO:      Started server process [1]
# INFO:      Application startup complete.
# INFO:      Uvicorn running on http://0.0.0.0:8000
```

Test it from a new terminal window

3

```
# Open a NEW terminal (first one is occupied by the running container)

# Test health endpoint
curl http://localhost:8000/health

# Expected: {"status":"healthy","service":"docmind",...}

# Test the auto-generated API docs
open http://localhost:8000/docs # opens in browser

# To stop the container: go back to first terminal and press Ctrl+C
```

Implementation 5 — docker-compose.yml

1

Create docker-compose.yml in your project root

Paste the full docker-compose content from Part 2, Section 2.6.

Run and verify

2

```
# Start both services (app + Redis)
docker-compose up

# In a NEW terminal, check both are running
docker-compose ps

# Should show:
# NAME                STATUS    PORTS
# docmind-app-1       Up        0.0.0.0:8000->8000/tcp
# docmind-redis-1     Up        0.0.0.0:6379->6379/tcp

# Test the API is responding
curl http://localhost:8000/health

# Stop everything cleanly
docker-compose down
```

Implementation 6 — Final Commit

Commit all new files

1

```
git add Dockerfile .dockerignore docker-compose.yml
git add .env.example main.py requirements.txt

git commit -m 'feat: dockerize DocMind with compose, health endpoint,
.env handling'

# Push your feature branch to GitHub
git push -u origin feature/dockerize
```

Verify .env was NOT committed

2

```
# List files in the last commit
git show --stat HEAD

# .env should NOT appear in this list
# .env.example SHOULD appear
# Dockerfile SHOULD appear
```


3

Tag this milestone

```
git tag -a v1.0 -m 'DocMind dockerized. Health endpoint. Compose working.'  
git push origin v1.0
```

Part 4: Verification and Troubleshooting

4.1 End-of-Day Checklist

Task	How to verify	Status
<code>.gitignore</code> created	<code>git status</code> — <code>.env</code> should not appear	
<code>.env.example</code> committed	<code>git show --stat HEAD</code> — <code>.env.example</code> appears	
<code>.env</code> NOT committed	<code>git show --stat HEAD</code> — <code>.env</code> does NOT appear	
Dockerfile builds	<code>docker build -t docmind:v1 .</code> ends with 'Successfully built'	
<code>.dockerignore</code> created	<code>docker run --rm docmind:v1 ls /app</code> — <code>.env</code> not listed	
Container runs on port 8000	<code>curl localhost:8000/health</code> returns JSON response	
<code>/health</code> endpoint works	Response contains <code>{"status": "healthy"}</code>	
<code>docker-compose.yml</code> created	<code>docker-compose up</code> starts both app and Redis	
Feature branch pushed	GitHub shows <code>feature/dockerize</code> branch	
<code>v1.0</code> tag created	<code>git tag</code> shows <code>v1.0</code>	

4.2 Troubleshooting Guide

Error	Cause	Fix
COPY failed: no such file requirements.txt	<code>requirements.txt</code> doesn't exist	Run: <code>pip freeze > requirements.txt</code> first
Port 8000 already allocated	Another process (native uvicorn?) using port 8000	Kill it with: <code>lsof -ti:8000 xargs kill</code> , or use <code>-p 8001:8000</code>
ModuleNotFoundError inside container	Library not in <code>requirements.txt</code>	Add to <code>requirements.txt</code> , rebuild: <code>docker build -t docmind:v1 .</code>
Cannot connect to Docker daemon	Docker Desktop not running	Open Docker Desktop, wait for whale icon
No module named 'main'	CMD points to wrong filename	Change CMD in Dockerfile to match your actual filename
.env not found / KeyError	Running without <code>--env-file</code> flag	Add <code>--env-file .env</code> to docker run command
Connection refused on localhost:8000	Missing <code>--host 0.0.0.0</code> in CMD	Add <code>--host 0.0.0.0 --port 8000</code> to your uvicorn CMD

4.3 Quick Concept Quiz — Say These Out Loud Before Tomorrow

Question	Answer
Difference between image and container?	Image = frozen snapshot (recipe result). Container = running instance of that image.
Why COPY requirements.txt before COPY . . ?	Layer caching. Code changes often; if reqs.txt unchanged, pip install is skipped.
Why does .env go in .dockerignore AND .gitignore?	Git: prevents public GitHub exposure. Docker: prevents API keys being baked into the image.
What does -p 8000:8000 mean?	Map host port 8000 to container port 8000. Format is host:container.
Why --host 0.0.0.0 in CMD?	Without it, the API only accepts connections from inside the container. Load balancers can't reach it.
What does docker-compose solve?	Starts multiple services together with one command and sets up networking between them.
What is a Git tag?	A named bookmark on a commit. Like a version label: v1.0, v1.1.

The End.